Agent Design Document

Student Information Management System (SIMS) — AI-Assisted Laravel Development Project

Project: Student Information Management System (SIMS)
Course: Laravel Web Development
Student: Rexon Timbal
Date: November 1, 2026

Objective

Demonstrate how AI agents were organized to complete the project: a deliberate two-role split between planning/verification and execution, rather than a single agent that both writes and grades its own code.

Architecture Used

```
Student Developer (Rexon Timbal)
   |
   v
Planner / Reviewer Agent  (Claude, chat session)
   |
   |---- drafts and reasons through each prompt before it is sent
   |---- holds the full handbook + rubric requirements in context
   |---- independently re-verifies every reported result
   |
   v
Builder Agent  (Claude Code, VS Code extension)
   |---- executes Artisan commands, writes migrations/controllers/views
   |---- runs the local terminal, reads/writes the filesystem directly
   |---- reports back what it did and what it tested
```

Why a Simpler Two-Role Split Was Sufficient

The handbook's example architecture (Planner → Database / Laravel / Testing / Documentation agents) models a project with several independent, parallel subsystems. SIMS is a single cohesive CRUD application built by one developer over a fixed two-week scope, where almost every module (Student, Subject, Enrollment) follows the exact same migration → model → controller → view pipeline built in Step 5 and reused, not reinvented, in Steps 6 and 7. Running four separate specialist agents in parallel for that kind of repetitive, sequential work would have added coordination overhead (merging each agent's output, resolving conflicting assumptions about the schema) without a corresponding benefit, since there was no real parallelism to exploit — each module genuinely depended on the one before it (Enrollment needs Student and Subject to exist first).

What the project did need, given it was graded partly on AI-output trustworthiness (rubric section C/D), was a hard separation between "the agent that writes code" and "the agent that checks the code is correct." That is the one specialization that mattered here, so the two-role split captures the single axis of risk this project actually had, instead of adding agent roles the project had no use for.

Roles and Responsibilities

Role	Responsible for
Planner / Reviewer (Claude chat)	Requirements analysis against the handbook/rubric; drafting and refining each prompt before it was run; independently re-verifying results by reading files, routes, and the database directly; deciding when a step needed a follow-up fix (e.g. Step 10's authorization gap, caught by review, not by the builder).
Builder (Claude Code, VS Code)	Migrations and schema design; controllers and CRUD operations; Blade views and routes; running the test suite; self-reporting issues it found while executing (e.g. the Step 9 route-name collision it caught and fixed mid-task).

Context Management

How context was provided:

- **Module-based context** — each of the 14 build steps scoped a single module or concern (Student, Subject, Enrollment, Dashboard, API, Auth polish, etc.), with the prompt for that step containing only what was needed for that module, not the whole app history.
- **Pattern reuse instead of re-explaining** — Steps 6 and 7 explicitly told the builder to follow the same rules()/controller/view conventions established in Step 5, rather than re-describing the whole CRUD pattern from scratch each time.
- **A running log as external memory** — PROJECT_LOG.md captured each step's prompt and verified result, so later steps (and this document) could reference exactly what had already been built without needing it restated in-context.

Token Optimization Strategies

- **Task decomposition** — one module/concern per prompt across 14 discrete steps, instead of one giant "build the whole system" prompt that would need to hold the entire app in context at once.
- **Agent specialization** — the builder session only needed the current module's files in context; the reviewer session only needed routes/DB state to verify, not the builder's full working memory.
- **Incremental generation** — each step built directly on the previous step's committed, working code (21 commits total) rather than regenerating or re-pasting prior modules into new prompts.